

- Description

This module manages a smart cold storage temperature monitoring and cooling control system, ensuring that:

1. **The temperature inside the cold storage is continuously monitored to prevent spoilage of stored goods.**
2. **If the temperature rises above the maximum safe threshold, the cooling system is activated.**
3. **If the temperature drops below the minimum threshold, the cooling system is deactivated to save energy.**
4. **A log of all temperature readings, cooling system activations, and shutdowns is maintained for operational efficiency.**

This system helps maintain product freshness, reduce energy consumption, and automate climate control in refrigerated storage facilities.

- Specification

module SmartColdStorageControlEx

var currentTemperature, maxTemperature, minTemperature: real

var coolingSystemActive: boolean

process updateTemperature(changeAmount: real)

functional scenario form:

currentTemperature <= maxTemperature && Storage temperature maintained.

||

currentTemperature > maxTemperature && coolingSystemActive = true,

Cooling system activated.

||

currentTemperature < minTemperature && coolingSystemActive = false,

Cooling system deactivated.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
```

```

import java.util.List;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c128_SmartColdStorageSystem
 * Domain: Intelligent cold storage (refrigeration) control
 *
 * Features:
 * - Core SmartColdStorage logic (two-
threshold cooling control with hysteresis-like behavior)
 * - Sensors: internal temperature, ambient temperature, humidity, doo
r state, load (heat influx)
 * - Controller: CoolingController (ON/OFF), energy usage/cost estimat
ion, cycle counter
 * - Alert system & file-backed logger
 * - Swing GUI: live status, thresholds, operations, simulation, alert
s, log tail
 * - Simulation engine: ambient & humidity drift, door events, load he
at, compressor dynamics
 * - Replay engine: parse weather/sim steps from log and replay
 * - CLI harness: VT/FT/Z3/UVT test groups (comment-to-
toggle single group)
 *
 * Compile: javac code/c128_SmartColdStorageSystem.java
 * Run GUI: java code.c128_SmartColdStorageSystem
 * Run CLI: java code.c128_SmartColdStorageSystem --cli
 * =====
 */
/* ===== Utilities ===== */
class CStr {
    static String fmt2(double v) { return new DecimalFormat("0.00").for
mat(v); }
    static String fmt3(double v) { return new DecimalFormat("0.000").fo
rmat(v); }
    static String now() { return LocalDateTime.now().format(DateTimeFor
matter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
}
/* ===== Logger ===== */
class CDataLogger {
    private final File file;
    private final ConcurrentLinkedQueue<String> inMemory = new Concurr
entLinkedQueue<>();
    private final int cap;

```

```

CDataLogger(String path, int inMemoryCap) {
    this.file = new File(path);
    this.cap = Math.max(100, inMemoryCap);
    try {
        if (!file.exists()) {
            File parent = file.getParentFile();
            if (parent != null) parent.mkdirs();
            file.createNewFile();
        }
    } catch (IOException e) {
        System.err.println("[Logger] Create file failed: " + e.getMessage());
    }
}

synchronized void log(String message) {
    String line = "[" + CStr.now() + "] " + message;
    inMemory.add(line);
    while (inMemory.size() > cap) inMemory.poll();
    try (FileOutputStream fos = new FileOutputStream(file, true);
        OutputStreamWriter osw = new OutputStreamWriter(fos, StandardCharsets.UTF_8);
        BufferedWriter bw = new BufferedWriter(osw)) {
        bw.write(line);
        bw.newLine();
    } catch (IOException e) {
        System.err.println("[Logger] Write failed: " + e.getMessage());
    }
}

List<String> recent(int n) {
    ArrayList<String> list = new ArrayList<>(inMemory);
    if (list.size() <= n) return list;
    return list.subList(list.size() - n, list.size());
}

File getFile() { return file; }
}

/* ===== Alerts ===== */
class CalertSystem {
    enum Severity { INFO, WARNING, CRITICAL }
    static class Alert {
        final Severity severity;
        final String message;
        final LocalDateTime time = LocalDateTime.now();
    }
}

```

```

        Alert(Severity s, String m) { this.severity = s; this.message =
m; }

        @Override public String toString() { return "[" + time + "]" +
severity + " " + message; }
    }
    private final List<Alert> alerts = new ArrayList<>();
    private final CDataLogger logger;
    CAlertSystem(CDataLogger logger) { this.logger = logger; }
    void push(Severity s, String msg) {
        Alert a = new Alert(s, msg);
        alerts.add(a);
        logger.log("ALERT " + s + " :: " + msg);
        System.out.println(a);
    }
    List<Alert> recent(int n) {
        if (alerts.size() <= n) return new ArrayList<>(alerts);
        return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
    }
}

/* ===== Sensors & Context =====
= */
abstract class CSensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    CSensor(String name) { this.name = name; }
    abstract void update(ColdContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}

class ColdContext {
    public double currentTemperature; // °C
    public double minTemperature;    // °C
    public double maxTemperature;    // °C
    public boolean coolingActive;
    public double ambientTemperature; // °C
    public double humidity;           // %
    public boolean doorOpen;          // true if door is open
    public double loadHeat;           // W equivalent (arbitrary)
}

```

```

    public double compressorPowerKw; // kW
    public int compressorCycles;     // count
    ColdContext copy() {
        ColdContext c = new ColdContext();
        c.currentTemperature = currentTemperature;
        c.minTemperature = minTemperature;
        c.maxTemperature = maxTemperature;
        c.coolingActive = coolingActive;
        c.ambientTemperature = ambientTemperature;
        c.humidity = humidity;
        c.doorOpen = doorOpen;
        c.loadHeat = loadHeat;
        c.compressorPowerKw = compressorPowerKw;
        c.compressorCycles = compressorCycles;
        return c;
    }
}

class InternalTempSensor extends CSensor {
    private double drift = 0.0;
    InternalTempSensor() { super("InternalTempSensor"); }
    @Override
    void update(ColdContext ctx) {
        drift += (rnd.nextDouble() - 0.5) * 0.01;
        value = ctx.currentTemperature + drift + (rnd.nextDouble() - 0.
5) * 0.05;
        ok = (value > -50 && value < 50);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class AmbientTempSensor128 extends CSensor {
    AmbientTempSensor128() { super("AmbientTempSensor"); }
    @Override
    void update(ColdContext ctx) {
        value = ctx.ambientTemperature + (rnd.nextDouble() - 0.5) * 0.1
;
        ok = (value > -50 && value < 60);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class HumiditySensorCS extends CSensor {
    HumiditySensorCS() { super("HumiditySensor"); }
    @Override
    void update(ColdContext ctx) {

```

```

        value = Math.max(0, Math.min(100, ctx.humidity + (rnd.nextDouble() - 0.5) * 1.5));
        ok = (value >= 0 && value <= 100);
        status = ok ? "OK" : "BROKEN";
    }
}

class DoorSensor extends CSensor {
    private boolean open;
    DoorSensor() { super("DoorSensor"); }
    @Override
    void update(ColdContext ctx) {
        open = ctx.doorOpen;
        value = open ? 1.0 : 0.0;
        ok = true;
        status = open ? "OPEN" : "CLOSED";
    }
    boolean isOpen() { return open; }
}

class LoadSensor extends CSensor {
    LoadSensor() { super("LoadSensor"); }
    @Override
    void update(ColdContext ctx) {
        value = Math.max(0, ctx.loadHeat + (rnd.nextDouble() - 0.5) * 1
0.0);
        ok = true;
        status = value > 80 ? "HIGH" : (value > 30 ? "MEDIUM" : "LOW");
    }
}

/* ===== Controller ===== */
class CoolingController {
    private boolean active = false;
    private double onAbove; // above max -> ON
    private double offBelow; // below min -> OFF (between => keep state
)

    private double compressorKw = 1.1; // rated power kW
    private double energyKWh = 0.0;
    private int cycles = 0;
    private boolean lastActive = false;
    CoolingController(double maxTemperature, double minTemperature) {
        this.onAbove = Math.max(maxTemperature, minTemperature + 0.5);
        this.offBelow = Math.min(minTemperature, maxTemperature - 0.5);
    }
    void update(double currentTemperature) {
        if (currentTemperature > onAbove) active = true;

```

```

        else if (currentTemperature < offBelow) active = false;
        if (active != lastActive) {
            cycles++;
            lastActive = active;
        }
    }
    void addTimeStep(double seconds) {
        if (active) energyKWh += compressorKw * (seconds / 3600.0);
    }
    void setThresholds(double maxT, double minT) {
        this.onAbove = Math.max(maxT, minT + 0.5);
        this.offBelow = Math.min(minT, maxT - 0.5);
    }
    boolean isActive() { return active; }
    void forceOn() { active = true; }
    void forceOff() { active = false; }
    double getEnergyKWh() { return energyKWh; }
    void resetEnergy() { energyKWh = 0.0; cycles = 0; }
    int getCycles() { return cycles; }
    double getOnAbove() { return onAbove; }
    double getOffBelow() { return offBelow; }
    double getCompressorKw() { return compressorKw; }
}
/* ===== Core Domain ===== */
class SmartColdStorage {
    private final CDataLogger logger;
    private final CAlertSystem alert;
    private double maxTemperature; // °C
    private double minTemperature; // °C
    private double currentTemperature; // °C
    // environment & usage
    private double ambientTemperature = 25.0; // °C
    private double humidity = 50.0; // %
    private boolean doorOpen = false;
    private double loadHeat = 20.0; // arbitrary W-like
    // components
    private final CoolingController controller;
    private final InternalTempSensor internalTempSensor = new InternalTempSensor();
    private final AmbientTempSensor128 ambientTempSensor = new AmbientTempSensor128();
    private final HumiditySensorCS humiditySensor = new HumiditySensorCS();
    private final DoorSensor doorSensor = new DoorSensor();

```

```

    private final LoadSensor loadSensor = new LoadSensor();
    private final List<Runnable> listeners = new ArrayList<>();
    SmartColdStorage(double maxTemperature, double minTemperature, double
initialTemperature,
                    CDataLogger logger, CAlertSystem alert) {
        this.maxTemperature = maxTemperature;
        this.minTemperature = minTemperature;
        this.currentTemperature = initialTemperature;
        this.logger = logger;
        this.alert = alert;
        this.controller = new CoolingController(maxTemperature, minTemp
erature);
        logger.log("SmartColdStorage initialized. Temp=" + CStr.fmt2(cu
rrentTemperature) + "°C " +
                    "(min=" + CStr.fmt2(minTemperature) + ", max=" + CStr.f
mt2(maxTemperature) + ")");
    }
    void addListener(Runnable r) { listeners.add(r); }
    private void emit() { for (Runnable r : listeners) try { r.run(); }
catch (Throwable ignored) {} }
    /* ===== Public API (original updateTemperature) ===== */
    public synchronized void updateTemperature(double changeAmount) {
        currentTemperature += changeAmount;
        manageCoolingSystem();
        sampleSensors();
        String s = "Current Storage Temperature: " + CStr.fmt2(currentT
emperature) + "°C | Cooling: " +
                    (controller.isActive() ? "ON" : "OFF");
        logger.log(s);
        System.out.println(s);
        if (controller.isActive()) System.out.println("Cooling System A
ctivated.");
        else System.out.println("Cooling System Deactivated.");
        emit();
    }
    /* ===== Control ===== */
    private void manageCoolingSystem() {
        controller.update(currentTemperature);
        // Alerts at threshold crossings
        if (currentTemperature > maxTemperature) {
            alert.push(CAlertSystem.Severity.WARNING,
                        "Above MAX threshold: " + CStr.fmt2(currentTemperat
ure) + "°C > " + CStr.fmt2(maxTemperature) + "°C");
        }
    }

```

```

        if (currentTemperature < minTemperature) {
            alert.push(CAlertSystem.Severity.INFO,
                "Below MIN threshold: " + CStr.fmt2(currentTemperature) + "°C < " + CStr.fmt2(minTemperature) + "°C");
        }
        // Door-open + high ambient + high humidity -> risk
        if (doorOpen && ambientTemperature > 28 && humidity > 70) {
            alert.push(CAlertSystem.Severity.CRITICAL,
                "Door OPEN with hot & humid ambient. Risk of rapid temperature rise.");
        }
    }
    /* ===== Sensors ===== */
    private void sampleSensors() {
        ColdContext ctx = buildContext();
        internalTempSensor.update(ctx);
        ambientTempSensor.update(ctx);
        humiditySensor.update(ctx);
        doorSensor.update(ctx);
        loadSensor.update(ctx);
        // Additional health checks
        if (!internalTempSensor.isOk()) {
            alert.push(CAlertSystem.Severity.CRITICAL, "Internal temperature sensor failure.");
        }
        if (controller.isActive() && currentTemperature < (minTemperature - 5)) {
            alert.push(CAlertSystem.Severity.WARNING, "Possible overcooling detected.");
        }
    }
    ColdContext buildContext() {
        ColdContext c = new ColdContext();
        c.currentTemperature = currentTemperature;
        c.minTemperature = minTemperature;
        c.maxTemperature = maxTemperature;
        c.coolingActive = controller.isActive();
        c.ambientTemperature = ambientTemperature;
        c.humidity = humidity;
        c.doorOpen = doorOpen;
        c.loadHeat = loadHeat;
        c.compressorPowerKw = controller.getCompressorKw();
        c.compressorCycles = controller.getCycles();
        return c;
    }

```

```

    }
    /* ===== Simulation hooks ===== */
    synchronized void tickSeconds(double seconds) {
        // Cooling effect: when active, pull temperature down; else, drift
        // toward ambient with load heat and door state.
        double coolingEffect = controller.isActive() ? -
0.08 : 0.0; // per second delta (coarse)
        double influx = 0.02 + (ambientTemperature - currentTemperature
) * 0.003
            + (doorOpen ? 0.05 : 0.0)
            + (loadHeat * 0.0002);
        currentTemperature += coolingEffect + influx;
        controller.addTimeStep(seconds);
        // Humidity dynamics: door open and ambient humidity raise inte
        // rnal humidity slowly
        if (doorOpen) humidity = clamp(humidity + 0.2, 0, 100);
        else humidity = clamp(humidity - 0.05, 0, 100);
        // After physics update, re-evaluate controller and sensors
        manageCoolingSystem();
        sampleSensors();
        emit();
    }
    synchronized void applyEnvironment(double ambientDelta, double humi
dityDelta,
                                     boolean doorFlag, double loadDel
ta) {
        ambientTemperature += ambientDelta;
        humidity = clamp(humidity + humidityDelta, 0, 100);
        doorOpen = doorFlag;
        loadHeat = Math.max(0, loadHeat + loadDelta);
        // Ambient impacts current temperature drift a bit
        currentTemperature += (ambientTemperature - currentTemperature)
* 0.002;
        manageCoolingSystem();
        sampleSensors();
        logger.log(String.format(Locale.US,
            "Env step: dAmb=%.3f°C, dHum=%.3f%, door=%s, dLoad=%.3
f | T=%.2f°C, Amb=%.2f°C, Hum=%.1f%, Load=%.1f",
            ambientDelta, humidityDelta, String.valueOf(doorFlag),
loadDelta,
            currentTemperature, ambientTemperature, humidity, loadH
eat));
        emit();
    }
}

```

```

/* ===== Maintenance ===== */
synchronized void calibrateTemperature(double observed) {
    double old = currentTemperature;
    currentTemperature = observed;
    logger.log("Calibration: storageT " + CStr.fmt2(old) + " -> " +
CStr.fmt2(currentTemperature));
    sampleSensors();
    emit();
}

synchronized void setThresholds(double maxT, double minT) {
    double oldMax = maxTemperature, oldMin = minTemperature;
    // Keep a small mandatory gap to avoid flip-flop
    maxTemperature = Math.max(minT + 0.5, maxT);
    minTemperature = Math.min(minT, maxTemperature - 0.5);
    controller.setThresholds(maxTemperature, minTemperature);
    logger.log("Thresholds changed: max " + CStr.fmt2(oldMax) + "->
" + CStr.fmt2(maxTemperature) +
        ", min " + CStr.fmt2(oldMin) + "->" + CStr.fmt2(minTemp
erature));
    sampleSensors();
    emit();
}

/* ===== Getters for GUI/tests ===== */
double getCurrentTemperature() { return currentTemperature; }
double getMaxTemperature() { return maxTemperature; }
double getMinTemperature() { return minTemperature; }
boolean isCoolingActive() { return controller.isActive(); }
double getAmbientTemperature() { return ambientTemperature; }
double getHumidity() { return humidity; }
boolean isDoorOpen() { return doorOpen; }
double getLoadHeat() { return loadHeat; }
double getEnergyKWh() { return controller.getEnergyKWh(); }
int getCompressorCycles() { return controller.getCycles(); }
InternalTempSensor internalSensor() { return internalTempSensor; }
AmbientTempSensor128 ambientSensor() { return ambientTempSensor; }
HumiditySensorCS humiditySensor() { return humiditySensor; }
DoorSensor doorStateSensor() { return doorSensor; }
LoadSensor loadSensor() { return loadSensor; }
private static double clamp(double v, double lo, double hi) { retur
n Math.max(lo, Math.min(hi, v)); }
}

/* ===== Simulation Engine =====
= */
class ColdSimulationEngine implements Runnable {

```

```

private final SmartColdStorage storage;
private final CDataLogger logger;
private final AtomicBoolean running = new AtomicBoolean(false);
private Thread thread;
private volatile double ambientDrift = +0.02; // °C per tick
private volatile double humidityDrift = +0.08; // % per tick
private volatile double loadDrift = +0.5; // Load per tick
private volatile double tickSeconds = 1.0;
private volatile long sleepMillis = 250L;
private volatile boolean randomize = true;
private final Random rnd = new Random();
ColdSimulationEngine(SmartColdStorage storage, CDataLogger logger)
{
    this.storage = storage;
    this.logger = logger;
}
void start() {
    if (running.get()) return;
    running.set(true);
    thread = new Thread(this, "ColdSimEngine");
    thread.setDaemon(true);
    thread.start();
    logger.log("Cold simulation started.");
}
void stop() {
    running.set(false);
    if (thread != null) {
        try { thread.join(2000); } catch (InterruptedException ignored) {}
    }
    logger.log("Cold simulation stopped.");
}
boolean isRunning() { return running.get(); }
void setParams(double ambDrift, double humDrift, double loadDrift,
               double tickSeconds, long delayMs, boolean randomize)
{
    this.ambientDrift = ambDrift;
    this.humidityDrift = humDrift;
    this.loadDrift = loadDrift;
    this.tickSeconds = Math.max(0.1, tickSeconds);
    this.sleepMillis = Math.max(20, delayMs);
    this.randomize = randomize;
    logger.log(String.format(Locale.US,

```

```

        "Sim params updated: dAmb=%.4f, dHum=%.4f, dLoad=%.4f,
dt=%.2fs, delay=%dms, random=%s",
        ambDrift, humDrift, loadDrift, this.tickSeconds, this.sleepMillis, String.valueOf(this.randomize)));
    }
    @Override
    public void run() {
        while (running.get()) {
            double dAmb = ambientDrift;
            double dHum = humidityDrift;
            double dLoad = loadDrift;
            boolean door = false;
            if (randomize) {
                dAmb += (rnd.nextDouble() - 0.5) * Math.abs(ambientDrift) * 1.6;
                dHum += (rnd.nextDouble() - 0.5) * Math.abs(humidityDrift) * 2.0;
                dLoad = Math.max(-2.0, dLoad + (rnd.nextDouble() - 0.5) * 2.5);
                door = rnd.nextDouble() < 0.12; // occasional door openings
            }
            storage.applyEnvironment(dAmb, dHum, door, dLoad);
            storage.tickSeconds(tickSeconds);
            try { Thread.sleep(sleepMillis); }
            catch (InterruptedException e) { break; }
        }
    }
}
/* ===== Replay (History) =====
*/
class ColdReplayEngine {
    private final SmartColdStorage storage;
    private final CDataLogger logger;
    ColdReplayEngine(SmartColdStorage storage, CDataLogger logger) {
        this.storage = storage;
        this.logger = logger;
    }
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
    }
}

```

```

        logger.log("Replay started from: " + logFile.getAbsolutePath())
;
        try (BufferedReader br = new BufferedReader(new InputStreamReader(
new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.contains("Env step:")) continue;
                try {
                    int idx = line.indexOf("Env step:");
                    String seg = line.substring(idx);
                    // Format: Env step: dAmb=..., dHum=..., door=...,
dLoad=... | ...
                    String[] parts = seg.split("\\|")[0].split(",");
                    double dAmb = Double.parseDouble(parts[0].split("=")
)[1].replace("°C", "").trim());
                    double dHum = Double.parseDouble(parts[1].split("=")
)[1].replace("%", "").trim());
                    boolean door = parts[2].split("=")[1].trim().equals
IgnoreCase("true");
                    double dLoad = Double.parseDouble(parts[3].split("=")
)[1].trim());
                    storage.applyEnvironment(dAmb, dHum, door, dLoad);
                    storage.tickSeconds(1.0);
                    Thread.sleep(delayMs);
                } catch (Throwable ignored) {}
            }
        } catch (IOException e) {
            System.err.println("[Replay] Error: " + e.getMessage());
        }
        logger.log("Replay finished.");
    }
}

/* ===== GUI ===== */
class ColdGUI extends JFrame {
    private final SmartColdStorage storage;
    private final CDataLogger logger;
    private final CAlertSystem alerts;
    private final ColdSimulationEngine engine;
    private final ColdReplayEngine replay;
    private JLabel lblT, lblMax, lblMin, lblCool, lblAmb, lblHum, lblDo
or, lblLoad, lblEnergy, lblCycles;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JTextArea txtLog;

```

```

private JTextArea txtAlerts;
private JSpinner spMax, spMin;
private JSpinner spAmb, spHum, spLoad, spDt, spDelay;
private JCheckBox cbRandomize, cbDoor;
private JButton btnStart, btnStop, btnReplay, btnCalib, btnAdd, btn
Sub, btnResetEnergy;
private javax.swing.Timer uiTimer;
ColdGUI(SmartColdStorage storage, CDataLogger logger, CAlertSystem
alerts,
        ColdSimulationEngine engine, ColdReplayEngine replay) {
    super("SmartColdStorage - Intelligent Cooling Control");
    this.storage = storage;
    this.logger = logger;
    this.alerts = alerts;
    this.engine = engine;
    this.replay = replay;
    setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    buildUi();
    setSize(1180, 760);
    setLocationRelativeTo(null);
    uiTimer = new javax.swing.Timer(500, e -> refreshUi());
    uiTimer.start();
    storage.addListener(this::refreshUi);
}
private JPanel titled(String title, JComponent inner) {
    JPanel p = new JPanel(new BorderLayout());
    p.setBorder(BorderFactory.createTitledBorder(title));
    p.add(inner, BorderLayout.CENTER);
    return p;
}
private void buildUi() {
    JPanel root = new JPanel(new BorderLayout(10,10));
    root.setBorder(new EmptyBorder(10,10,10,10));
    setContentPane(root);
    /* LEFT: Status & Controls */
    JPanel left = new JPanel();
    left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
    JPanel status = new JPanel(new GridLayout(0,2,6,6));
    lblT = new JLabel("-");
    lblMax = new JLabel("-");
    lblMin = new JLabel("-");
    lblCool = new JLabel("-");
    lblAmb = new JLabel("-");
    lblHum = new JLabel("-");

```

```

        lblDoor = new JLabel("-");
        lblLoad = new JLabel("-");
        lblEnergy = new JLabel("-");
        lblCycles = new JLabel("-");
        status.add(new JLabel("Storage Temp (°C):")); status.add(lblT);
        status.add(new JLabel("Max (°C):")); status.add(lblMax);
;
        status.add(new JLabel("Min (°C):")); status.add(lblMin);
;
        status.add(new JLabel("Cooling:")); status.add(lblCool);
);
        status.add(new JLabel("Ambient (°C):")); status.add(lblAmb);
;
        status.add(new JLabel("Humidity (%):")); status.add(lblHum);
;
        status.add(new JLabel("Door:")); status.add(lblDoor);
;
        status.add(new JLabel("Load:")); status.add(lblLoad);
;
        status.add(new JLabel("Energy (kWh):")); status.add(lblEnergy);
);
        status.add(new JLabel("Cycles:")); status.add(lblCycles);
s);

        left.add(titled("Live Status", status));
        JPanel th = new JPanel(new GridLayout(0,2,6,6));
        spMax = new JSpinner(new SpinnerNumberModel(5.0, -
30.0, 30.0, 0.1));
        spMin = new JSpinner(new SpinnerNumberModel(-5.0, -
50.0, 20.0, 0.1));
        JButton btnApply = new JButton("Apply Thresholds");
        btnApply.addActionListener(e -> {
            double maxT = ((Number)spMax.getValue()).doubleValue();
            double minT = ((Number)spMin.getValue()).doubleValue();
            storage.setThresholds(maxT, minT);
        });
        th.add(new JLabel("Max temp (°C)")); th.add(spMax);
        th.add(new JLabel("Min temp (°C)")); th.add(spMin);
        th.add(new JLabel(" ")); th.add(btnApply);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Thresholds", th));
        JPanel ops = new JPanel(new GridLayout(0,2,6,6));
        btnAdd = new JButton("Raise Temp +1.0°C");
        btnSub = new JButton("Lower Temp -1.0°C");
        btnCalib = new JButton("Calibrate Temp");

```

```

        btnResetEnergy = new JButton("Reset Energy/Cycles");
        btnAdd.addActionListener(e -> storage.updateTemperature(+1.0));
        btnSub.addActionListener(e -> storage.updateTemperature(-1.0));
        btnCalib.addActionListener(e -> {
            String s = JOptionPane.showInputDialog(this, "Observed storage temp (°C):", CStr.fmt2(storage.getCurrentTemperature()));
            if (s == null) return;
            try { storage.calibrateTemperature(Double.parseDouble(s));
        }
            catch (NumberFormatException ex) { JOptionPane.showMessageDialog(this, "Invalid number."); }
        });
        btnResetEnergy.addActionListener(e -> {
            try {
                java.lang.reflect.Field fCtrl = SmartColdStorage.class.getDeclaredField("controller");
                fCtrl.setAccessible(true);
                CoolingController ctrl = (CoolingController) fCtrl.get(storage);

                ctrl.resetEnergy();
                logger.log("Energy/cycle counters reset by user.");
            } catch (Throwable ex) {
                JOptionPane.showMessageDialog(this, "Reset failed: " + ex.getMessage());
            }
        });
        ops.add(btnAdd); ops.add(btnSub);
        ops.add(btnCalib); ops.add(btnResetEnergy);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Operations", ops));
        JPanel sim = new JPanel(new GridLayout(0,2,6,6));
        spAmb = new JSpinner(new SpinnerNumberModel(+0.02, -5.0, 5.0, 0.01));
        spHum = new JSpinner(new SpinnerNumberModel(+0.08, -5.0, 5.0, 0.01));
        spLoad= new JSpinner(new SpinnerNumberModel(+0.50, -5.0, 5.0, 0.1));
        spDt = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0.1));
        spDelay = new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10));

        cbRandomize = new JCheckBox("Randomize", true);
        cbDoor = new JCheckBox("Force Door Open", false);
        btnStart = new JButton("Start Simulation");

```

```

        btnStop = new JButton("Stop Simulation");
        btnReplay= new JButton("Replay from Log");
        btnStart.addActionListener(e -> engine.setParams(
            ((Number)spAmb.getValue()).doubleValue(),
            ((Number)spHum.getValue()).doubleValue(),
            ((Number)spLoad.getValue()).doubleValue(),
            ((Number)spDt.getValue()).doubleValue(),
            ((Number)spDelay.getValue()).longValue(),
            cbRandomize.isSelected()
        ));
        btnStart.addActionListener(e -> engine.start());
        btnStop.addActionListener(e -> engine.stop());
        btnReplay.addActionListener(e -> new Thread(() -> replay.replay
(logger.getFile(), 60), "Replay").start());
        sim.add(new JLabel("ΔAmbient per tick (°C)")); sim.add(spAmb);
        sim.add(new JLabel("ΔHumidity per tick (%)")); sim.add(spHum);
        sim.add(new JLabel("ΔLoad per tick")); sim.add(spLoad);
        sim.add(new JLabel("dt (s)")); sim.add(spDt);
        sim.add(new JLabel("Delay (ms)")); sim.add(spDelay);
        sim.add(new JLabel(" ")); sim.add(cbRandomiz
e);
        sim.add(new JLabel(" ")); sim.add(cbDoor); /
/ manual door toggle for next click
        sim.add(btnStart); sim.add(btnStop);
        sim.add(new JLabel(" ")); sim.add(btnReplay)
;

        left.add(Box.createVerticalStrut(6));
        left.add(titled("Simulation", sim));
        getContentPane().add(left, BorderLayout.WEST);
        /* CENTER: Sensors + Alerts */
        JPanel center = new JPanel(new BorderLayout(6,6));
        tblModel = new DefaultTableModel(new String[]{"Sensor","Value",
"OK","Status"}, 0) {
            @Override public boolean isCellEditable(int row, int column
) { return false; }
        };
        tblSensors = new JTable(tblModel);
        JScrollPane spTbl = new JScrollPane(tblSensors);
        center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
        txtAlerts = new JTextArea(8, 60);
        txtAlerts.setEditable(false);
        center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
        getContentPane().add(center, BorderLayout.CENTER);

```

```

        /* RIGHT: Log tail */
        txtLog = new JTextArea();
        txtLog.setEditable(false);
        JScrollPane spLog = new JScrollPane(txtLog);
        getContentPane().add(titled("Log (tail)", spLog), BorderLayout.
EAST);

        // periodic alerts refresh
        javax.swing.Timer alertsTimer = new javax.swing.Timer(1000, e -
> {

            StringBuilder sb = new StringBuilder();
            for (CAAlertSystem.Alert a : alerts.recent(18)) {
                sb.append(a).append("\n");
            }
            txtAlerts.setText(sb.toString());
            txtAlerts.setCaretPosition(txtAlerts.getDocument().getLengt
h());
        });
        alertsTimer.start();
        // small door toggle hook using the checkbox
        cbDoor.addActionListener(e -> {
            boolean door = cbDoor.isSelected();
            storage.applyEnvironment(0.0, 0.0, door, 0.0);
        });
    }
    private void refreshUi() {
        lblT.setText(CStr.fmt2(storage.getCurrentTemperature()));
        lblMax.setText(CStr.fmt2(storage.getMaxTemperature()));
        lblMin.setText(CStr.fmt2(storage.getMinTemperature()));
        lblCool.setText(storage.isCoolingActive() ? "ON" : "OFF");
        lblCool.setForeground(storage.isCoolingActive() ? new Color(0,1
20,0) : Color.RED);
        lblAmb.setText(CStr.fmt2(storage.getAmbientTemperature()));
        lblHum.setText(CStr.fmt2(storage.getHumidity()));
        lblDoor.setText(storage.isDoorOpen() ? "OPEN" : "CLOSED");
        lblLoad.setText(CStr.fmt2(storage.getLoadHeat()));
        lblEnergy.setText(CStr.fmt2(storage.getEnergyKWh()));
        lblCycles.setText(Integer.toString(storage.getCompressorCycles(
)));
        tblModel.setRowCount(0);
        tblModel.addRow(new Object[]{
            storage.internalSensor().getName(),
            CStr.fmt2(storage.internalSensor().getValue()),
            storage.internalSensor().isOk(), storage.internalSensor
().getStatus()

```

```

    });
    tblModel.addRow(new Object[]{
        storage.ambientSensor().getName(),
        CStr.fmt2(storage.ambientSensor().getValue()),
        storage.ambientSensor().isOk(), storage.ambientSensor()
.getStatus()
    });
    tblModel.addRow(new Object[]{
        storage.humiditySensor().getName(),
        CStr.fmt2(storage.humiditySensor().getValue()),
        storage.humiditySensor().isOk(), storage.humiditySensor
().getStatus()
    });
    tblModel.addRow(new Object[]{
        storage.doorStateSensor().getName(),
        storage.doorStateSensor().getValue() > 0.5 ? "1 (open)"
: "0 (closed)",
        storage.doorStateSensor().isOk(), storage.doorStateSens
or().getStatus()
    });
    tblModel.addRow(new Object[]{
        storage.loadSensor().getName(),
        CStr.fmt2(storage.loadSensor().getValue()),
        storage.loadSensor().isOk(), storage.loadSensor().getSt
atus()
    });
    List<String> lines = logger.recent(220);
    StringBuilder sb = new StringBuilder();
    for (String s : lines) sb.append(s).append("\n");
    txtLog.setText(sb.toString());
    txtLog.setCaretPosition(txtLog.getDocument().getLength());
}
}
/* ===== CLI Harness ===== */
class ColdCLIHarness {
    private final SmartColdStorage storage;
    ColdCLIHarness(SmartColdStorage storage) { this.storage = storage;
}

    void runAll() {
        System.out.println("==== CLI Demo (VT) =====");
        vt();
        System.out.println("==== CLI Demo (FT) =====");
        ft();
        System.out.println("==== CLI Demo (Z3 mimic) =====");
    }
}

```

```

        z3();
        System.out.println("==== CLI Demo (UVT) =====");
        uvt();
    }
    // VT: mirrors user's VT to hit >max => ON, <min => OFF, and between
n => keep.
    void vt() {
        storage.updateTemperature(6.0);    // 0 -> 6 (>max) => ON
        storage.updateTemperature(-
4.0); // 6 -> 2 (between) => keep ON
        storage.updateTemperature(-7.0);   // 2 -> -5 (<min) => OFF
        storage.updateTemperature(-12.0);  // -5 -> -17 (<min) => OFF
        storage.updateTemperature(3.0);    // -17 -> -
14 (still <min) => OFF
    }
    // FT: extreme numeric edges as in user's FT.
    void ft() {
        storage.updateTemperature(1.6349721113176);
        storage.updateTemperature(-1.797693134862);
        storage.updateTemperature(1.0222176649217);
        storage.updateTemperature(1.0432843813458);
        storage.updateTemperature(0.0);
    }
    // Z3-like: same as VT for deterministic branch coverage.
    void z3() {
        storage.updateTemperature(6.0);
        storage.updateTemperature(-4.0);
        storage.updateTemperature(-7.0);
        storage.updateTemperature(-12.0);
        storage.updateTemperature(3.0);
    }
    // UVT: mirrors VT.
    void uvt() {
        storage.updateTemperature(6.0);
        storage.updateTemperature(-4.0);
        storage.updateTemperature(-7.0);
        storage.updateTemperature(-12.0);
        storage.updateTemperature(3.0);
    }
}
/* ===== Main ===== */
public class c128_SmartColdStorageSystem {
    public static void main(String[] args) {
        //         boolean forceCli = Arrays.asList(args).contains("--cli");

```

```

        boolean forceCli = true;
        CDataLogger logger = new CDataLogger("cold_log.txt", 800);
        CAlertSystem alertSystem = new CAlertSystem(logger);
        // Core: user's initial thresholds and temperature
        SmartColdStorage storage = new SmartColdStorage(5.0, -
5.0, 0.0, logger, alertSystem);
        // Engines
        ColdSimulationEngine engine = new ColdSimulationEngine(storage,
logger);
        ColdReplayEngine replay = new ColdReplayEngine(storage, logger)
;
        // ===== CLI branch (sequential 4 groups; comment 3 lines to ru
n single group) =====
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            ColdCLIHarness harness = new ColdCLIHarness(storage);
//            harness.vt(); // ← keep to run VT
            harness.ft(); // ← comment out to skip FT
//            harness.z3(); // ← comment out to skip Z3
//            harness.uvt(); // ← comment out to skip UVT
            // Optional extra ticks to exercise controller transitions
& alerts
            for (int i = 0; i < 20; i++) {
                storage.applyEnvironment(+0.05, +0.15, (i % 6 == 0), +0
.3);
                storage.tickSeconds(1.0);
                try { Thread.sleep(100L); } catch (Exception ignored) {
}
            }
            logger.log("CLI demo finished.");
            System.out.println("CLI demo finished. Log written to: " +
logger.getFile().getAbsolutePath());
            return;
        }
        // ===== GUI branch =====
        logger.log("Running in GUI mode.");
        SwingUtilities.invokeLater(() -> {
            ColdGUI gui = new ColdGUI(storage, logger, alertSystem, eng
ine, replay);
            gui.setVisible(true);
        });
    }
    private static boolean headless() {
        try { return GraphicsEnvironment.isHeadless(); }

```

```

        catch (Throwable t) { return true; }
    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c128_SmartColdStorageSystem.java	36.6 %	41	71	112
> CAAlertSystem	0.0 %	0	2	2
> ColdReplayEngine	0.0 %	0	8	8
> ColdSimulationEngine	0.0 %	0	10	10
> CDataLogger	25.0 %	2	6	8
> AmbientTempSensor128	50.0 %	3	3	6
> CoolingController	50.0 %	4	4	8
> HumiditySensorCS	50.0 %	3	3	6
> InternalTempSensor	50.0 %	3	3	6
> LoadSensor	50.0 %	2	2	4
> SmartColdStorage	53.6 %	15	13	28
> c128_SmartColdStorageSystem	62.5 %	5	3	8
▼ DoorSensor	100.0 %	4	0	4
▲ update(ColdContext)	100.0 %	4	0	4
▲ DoorSensor()		0	0	0

3. Vibration testing instruction coverage: 90.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c128_SmartColdStorageSystem.java	29.0 %	933	2,284	3,217
> ColdContext	6.1 %	3	46	49
> ColdReplayEngine	6.4 %	9	131	140
> ColdSimulationEngine	15.8 %	38	202	240
> CAAlertSystem	18.6 %	11	48	59
> ColdCLIHarness	24.3 %	27	84	111
> CoolingController	47.7 %	61	67	128
> CStr	54.5 %	12	10	22
> CDataLogger	57.9 %	70	51	121
> SmartColdStorage	68.3 %	402	187	589
> CSensor	69.0 %	20	9	29
> c128_SmartColdStorageSystem	69.8 %	81	35	116
> LoadSensor	89.7 %	35	4	39
> DoorSensor	90.3 %	28	3	31
> AmbientTempSensor128	95.0 %	38	2	40
> HumiditySensorCS	95.5 %	42	2	44

4. Fuzz testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c128_SmartColdStorageSystem.java	36.6 %	41	71	112
> CAAlertSystem	0.0 %	0	2	2
> ColdReplayEngine	0.0 %	0	8	8
> ColdSimulationEngine	0.0 %	0	10	10
> CDataLogger	25.0 %	2	6	8
> AmbientTempSensor128	50.0 %	3	3	6
> CoolingController	50.0 %	4	4	8
> HumiditySensorCS	50.0 %	3	3	6
> InternalTempSensor	50.0 %	3	3	6
> LoadSensor	50.0 %	2	2	4
> SmartColdStorage	53.6 %	15	13	28
> c128_SmartColdStorageSystem	62.5 %	5	3	8
> DoorSensor	100.0 %	4	0	4
> ColdCLIHarness		0	0	0
> ColdContext		0	0	0
> CSensor		0	0	0

5. Fuzz testing instruction coverage: 90.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c128_SmartColdStorageSystem.java		29.0 %	933	2,284	3,217
> ColdContext		6.1 %	3	46	49
> ColdReplayEngine		6.4 %	9	131	140
> ColdSimulationEngine		15.8 %	38	202	240
> CAlertSystem		18.6 %	11	48	59
> ColdCLIHarness		24.3 %	27	84	111
> CoolingController		47.7 %	61	67	128
> CStr		54.5 %	12	10	22
> CDataLogger		57.9 %	70	51	121
> SmartColdStorage		68.3 %	402	187	589
> CSensor		69.0 %	20	9	29
> c128_SmartColdStorageSystem		69.8 %	81	35	116
> LoadSensor		89.7 %	35	4	39
> DoorSensor		90.3 %	28	3	31
> AmbientTempSensor128		95.0 %	38	2	40
> HumiditySensorCS		95.5 %	42	2	44

6. Z3 branch coverage: 100%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c128_SmartColdStorageSystem.java		36.6 %	41	71	112
> CAlertSystem		0.0 %	0	2	2
> ColdReplayEngine		0.0 %	0	8	8
> ColdSimulationEngine		0.0 %	0	10	10
> CDataLogger		25.0 %	2	6	8
> AmbientTempSensor128		50.0 %	3	3	6
> CoolingController		50.0 %	4	4	8
> HumiditySensorCS		50.0 %	3	3	6
> InternalTempSensor		50.0 %	3	3	6
> LoadSensor		50.0 %	2	2	4
> SmartColdStorage		53.6 %	15	13	28
> c128_SmartColdStorageSystem		62.5 %	5	3	8
▼ DoorSensor		100.0 %	4	0	4

7. Z3 instruction coverage: 90.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c128_SmartColdStorageSystem.java		33.1 %	1,064	2,153	3,217
> ColdContext		6.1 %	3	46	49
> ColdReplayEngine		6.4 %	9	131	140
> ColdSimulationEngine		15.8 %	38	202	240
> ColdCLIHarness		24.3 %	27	84	111
> CStr		54.5 %	12	10	22
> CAlertSystem		55.9 %	33	26	59
> CDataLogger		57.9 %	70	51	121
> CoolingController		60.9 %	78	50	128
> CSensor		69.0 %	20	9	29
> c128_SmartColdStorageSystem		69.8 %	81	35	116
> SmartColdStorage		74.4 %	438	151	589
> LoadSensor		89.7 %	35	4	39
▼ DoorSensor		90.3 %	28	3	31
▲ DoorSensor()		100.0 %	4	0	4
▲ update(ColdContext)		100.0 %	24	0	24

8. Unimproved V-method branch coverage: 100%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c128_SmartColdStorageSystem.java		36.6 %	41	71	112
> CAlertSystem		0.0 %	0	2	2
> ColdReplayEngine		0.0 %	0	8	8
> ColdSimulationEngine		0.0 %	0	10	10
> CDataLogger		25.0 %	2	6	8
> AmbientTempSensor128		50.0 %	3	3	6
> CoolingController		50.0 %	4	4	8
> HumiditySensorCS		50.0 %	3	3	6
> InternalTempSensor		50.0 %	3	3	6
> LoadSensor		50.0 %	2	2	4
> SmartColdStorage		53.6 %	15	13	28
> c128_SmartColdStorageSystem		62.5 %	5	3	8
> DoorSensor		100.0 %	4	0	4
> ColdCLIHarness			0	0	0

9. Unimproved V-method instruction coverage: 90.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  c128_SmartColdStorageSystem.java		33.1 %	1,064	2,153	3,217
> 		6.1 %	3	46	49
> 		6.4 %	9	131	140
> 		15.8 %	38	202	240
> 		24.3 %	27	84	111
> 		54.5 %	12	10	22
> 		55.9 %	33	26	59
> 		57.9 %	70	51	121
> 		60.9 %	78	50	128
> 		69.0 %	20	9	29
> 		69.8 %	81	35	116
> 		74.4 %	438	151	589
> 		89.7 %	35	4	39
▼ 		90.3 %	28	3	31
▲ DoorSensor()		100.0 %	4	0	4
▲ update(ColdContext)		100.0 %	24	0	24